

COMPTE RENDU

TP 3 : Authentification Web avec Servlets

Pr. Mohamed CHERRADI
TP N° 2 – J2EE, Java EE (JEE) ou Jakarta EE
TDIA2-S4 / 2025-26

1

Introduction

Objectif et contexte du TP

Ce compte rendu présente la réalisation du **TP 3: Authentification Web JEE**.

L'objectif est de développer une application sécurisée composée de trois pages : **Signup** (inscription), **Login** (connexion) et **Home** (page protégée).

L'application utilise les **Sessions HTTP** pour maintenir l'état de connexion et un **Filtre (AuthFilter)** pour protéger les pages privées. Les données utilisateurs sont stockées en mémoire dans un **HashMap** Java (sans base de données réelle).

Session HTTP

Maintien de l'état utilisateur entre les requêtes

AuthFilter

Protection des pages privées (/home)

HashMap

Base de données en mémoire (username → password)

2

Architecture du Projet

Flux de navigation et composants

L'application **TP3Auth** suit une architecture MVC avec un filtre de sécurité. Le **AuthFilter** intercepte toutes les requêtes vers **/home** et vérifie la session.

URL	Servlet	Rôle	Méthode
/signup	SignupServlet.java	Inscription d'un nouvel utilisateur	GET + POST
/login	LoginServlet.java	Connexion + création de session	GET + POST
/home	HomeServlet.java	Page protégée (filtrée)	GET
/logout	LogoutServlet.java	Destruction de la session	GET
/home (filtre)	AuthFilter.java	Vérifie session avant /home	@WebFilter

Flux

Description

Non connecté → /home

AuthFilter détecte: pas de session → redirect /login

POST /signup

SignupServlet enregistre dans HashMap → redirect /login

POST /login

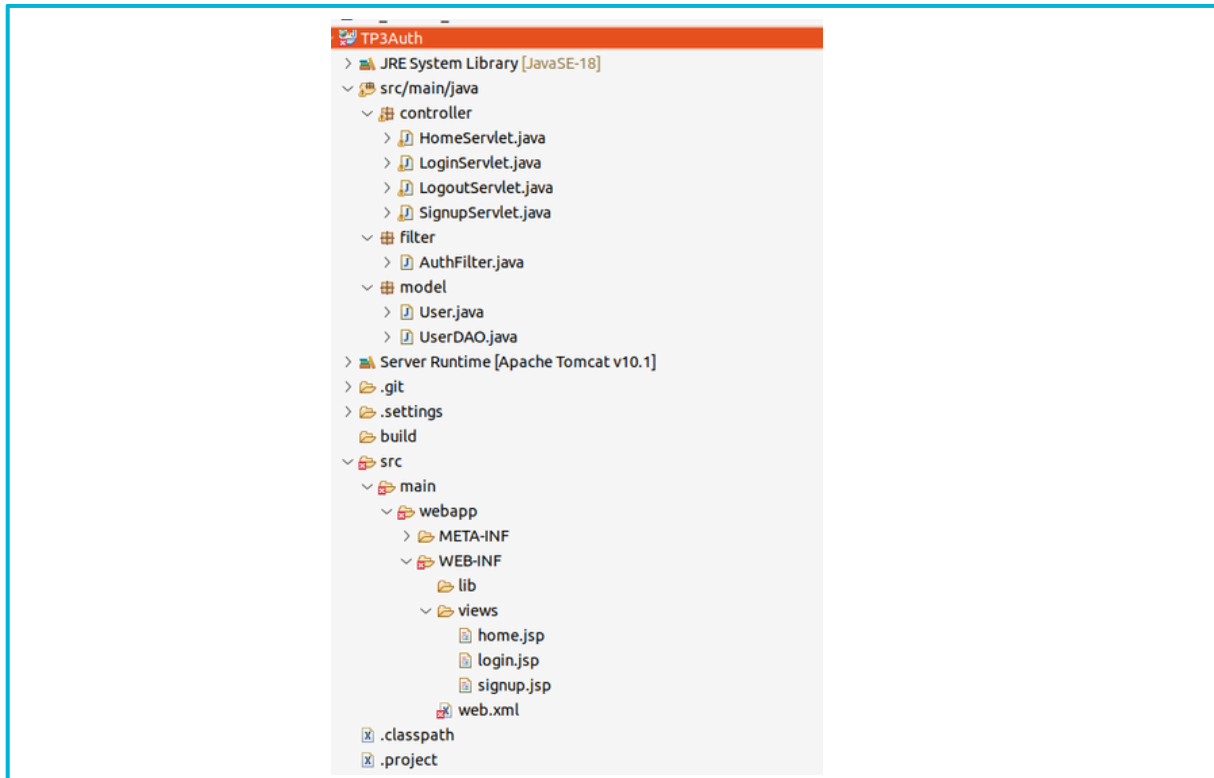
LoginServlet authentifie → session.setAttribute('username') → /home

GET /logout

LogoutServlet invalide la session → redirect /login

3 Structure du Projet

Le projet **TP3Auth** est un **Dynamic Web Project** Eclipse avec Tomcat 10.1. Les JSP sont sécurisées dans **WEB-INF/views/**.



- Structure du projet TP3Auth dans Eclipse IDE

Fichier	Emplacement	Rôle
UserDAO.java	src/main/java/model/	ArrayList username→password (BDD mémoire)
AuthFilter.java	src/main/java/filter/	Filtre @WebFilter('/home')
LoginServlet.java	src/main/java/controller/	@WebServlet('/login')
SignupServlet.java	src/main/java/controller/	@WebServlet('/signup')
HomeServlet.java	src/main/java/controller/	@WebServlet('/home')
LogoutServlet.java	src/main/java/controller/	@WebServlet('/logout')
login.jsp	WEB-INF/views/	Formulaire de connexion (privé)
signup.jsp	WEB-INF/views/	Formulaire d'inscription (privé)
home.jsp	WEB-INF/views/	Page d'accueil protégée (privé)
web.xml	WEB-INF/	Descripteur de déploiement

4.1 UserDao.java { La base de données en mémoire (ArrayList)}

```

import java.util.ArrayList;
import java.util.List;

public class UserDao {
    // In-memory store - same behaviour as your original UserStore
    private static final List<User> users = new ArrayList<>();

    // Authenticate: returns true if username + password match
    public static boolean authenticate(String username, String password) {
        for (User user : users) {
            if (user.getUsername().equals(username) &&
                user.getPassword().equals(password)) {
                return true;
            }
        }
        return false;
    }

    // Register a new user
    public static void register(String username, String password) {
        users.add(new User(username, password));
    }

    // Check if a username is already taken
    public static boolean userExists(String username) {
        for (User user : users) {
            if (user.getUsername().equals(username)) {
                return true;
            }
        }
        return false;
    }

    // Return total number of registered users
    public static int getUserCount() {
        return users.size();
    }
}

```

La `List<User>` stocke les utilisateurs sous forme d'objets de type `User` (clé = username, valeur = password encapsulés). Deux utilisateurs par défaut sont prédéfinis dans le bloc `static` : admin/1234 et hiba/password. Les méthodes `userExists()`, `authenticate()` et `register()` parcourent la liste avec une boucle `for` pour remplacer les requêtes SQL.

4.2 AuthFilter.java

```

2
3 import jakarta.servlet.*;
4 import jakarta.servlet.annotation.*;
5 import jakarta.servlet.http.*;
6 import java.io.IOException;
7
8 @WebFilter("/home")
9 public class AuthFilter implements Filter {
10
11     @Override
12     public void doFilter(ServletRequest request, ServletResponse response,
13                         FilterChain chain) throws IOException, ServletException {
14
15         HttpServletRequest req = (HttpServletRequest) request;
16         HttpServletResponse resp = (HttpServletResponse) response;
17
18         HttpSession session = req.getSession(false);
19         boolean loggedIn = (session != null && session.getAttribute("username") != null);
20
21         if (loggedIn) {
22             chain.doFilter(request, response);
23         } else {
24             resp.sendRedirect(req.getContextPath() + "/login");
25         }
26     }
27 }
28
29

```

Le filtre intercepte toutes les requêtes vers `/home` grâce à l'annotation `@WebFilter("/home")`. Si la session contient l'attribut `username`, la requête est transmise (`chain.doFilter()`). Sinon, l'utilisateur est redirigé vers `/login`.

4.3 LoginServlet.java

```
import controller;

jakarta.servlet.*;
jakarta.servlet.http.*;
jakarta.servlet.annotation.*;
java.io.IOException;
model.UserDAO;

@WebServlet("/login")
class LoginServlet extends HttpServlet {}

@Override
protected void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    request.getRequestDispatcher("/WEB-INF/views/login.jsp")
        .forward(request, response);
}

@Override
protected void doPost(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    request.setCharacterEncoding("UTF-8");
    String username = request.getParameter("username");
    String password = request.getParameter("password");

    if (username == null || password == null ||
        username.trim().isEmpty() || password.isEmpty()) {
        request.setAttribute("erreur", "Veuillez remplir tous les champs.");
        request.getRequestDispatcher("/WEB-INF/views/login.jsp")
            .forward(request, response);
        return;
    }

    if (UserDAO.authenticate(username.trim(), password)) {
        HttpSession session = request.getSession();
        session.setAttribute("username", username.trim());
        session.setMaxInactiveInterval(30 * 60);
        response.sendRedirect(request.getContextPath() + "/home");
    } else {
        request.setAttribute("erreur", "Nom d'utilisateur ou mot de passe incorrect.");
        request.getRequestDispatcher("/WEB-INF/views/login.jsp")
            .forward(request, response);
    }
}
```

- *LoginServlet.java : authentification + création de session*

La méthode **doPost()** récupère le username et password, appelle **UserDAO.authenticate()**, puis crée une session avec **session.setAttribute("username", ...)** et redirige vers **/home**. En cas d'échec, un message d'erreur est affiché.

4.4 SignupServlet.java

```
import jakarta.servlet.http.*;
import jakarta.servlet.annotation.*;
import java.io.IOException;
import model.UserDAO;

@WebServlet("/signup")
public class SignupServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        request.getRequestDispatcher("/WEB-INF/views/signup.jsp")
            .forward(request, response);
    }

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        request.setCharacterEncoding("UTF-8");
        String username = request.getParameter("username");
        String password = request.getParameter("password");
        String confirm = request.getParameter("confirm");

        if (username == null || username.trim().isEmpty() ||
            password == null || password.isEmpty() ||
            confirm == null || confirm.isEmpty()) {
            request.setAttribute("erreur", "Tous les champs sont obligatoires.");
            request.getRequestDispatcher("/WEB-INF/views/signup.jsp")
                .forward(request, response);
            return;
        }

        if (!password.equals(confirm)) {
            request.setAttribute("erreur", "Les mots de passe ne correspondent pas.");
            request.getRequestDispatcher("/WEB-INF/views/signup.jsp")
                .forward(request, response);
            return;
        }

        if (UserDAO.userExists(username.trim())) {
            request.setAttribute("erreur", "Ce nom d'utilisateur est déjà pris.");
            request.getRequestDispatcher("/WEB-INF/views/signup.jsp")
                .forward(request, response);
            return;
        }

        UserDAO.register(username.trim(), password);
        request.setAttribute("succes", "Compte créé ! Vous pouvez maintenant vous connecter.");
        request.getRequestDispatcher("/WEB-INF/views/login.jsp")
            .forward(request, response);
    }
}
```

- *SignupServlet.java : inscription avec validation*

4.5 HomeServlet.java

```
package controller;

import jakarta.servlet.*;
import jakarta.servlet.http.*;
import jakarta.servlet.annotation.*;
import java.io.IOException;
import model.UserDAO;

@WebServlet("/home")
public class HomeServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        HttpSession session = request.getSession(false);
        String username = (String) session.getAttribute("username");

        request.setAttribute("username", username);
        request.setAttribute("userCount", UserDAO.getUserCount());

        request.getRequestDispatcher("/WEB-INF/views/home.jsp")
            .forward(request, response);
    }
}
```

- *HomeServlet.java : récupération de la session et affichage*

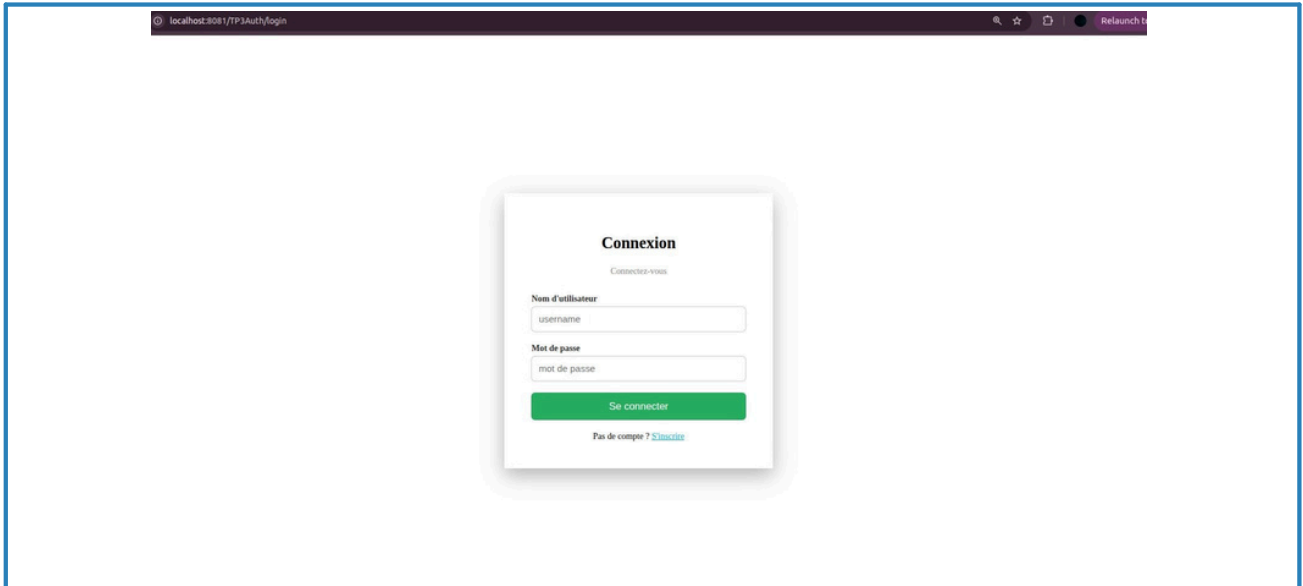
4.6 LogoutServlet.java

```
HomeServlet.java LogoutServlet.java X
1 package ma.ensah;
2
3 import jakarta.servlet.*;
4
5 @WebServlet("/logout")
6 public class LogoutServlet extends HttpServlet {
7
8     @Override
9     protected void doGet(HttpServletRequest request, HttpServletResponse response)
10         throws ServletException, IOException {
11
12         HttpSession session = request.getSession(false);
13         if (session != null) {
14             session.invalidate();
15         }
16         response.sendRedirect(request.getContextPath() + "/login");
17     }
18 }
19
20
21 }
```

- *LogoutServlet.java : invalidation de la session*

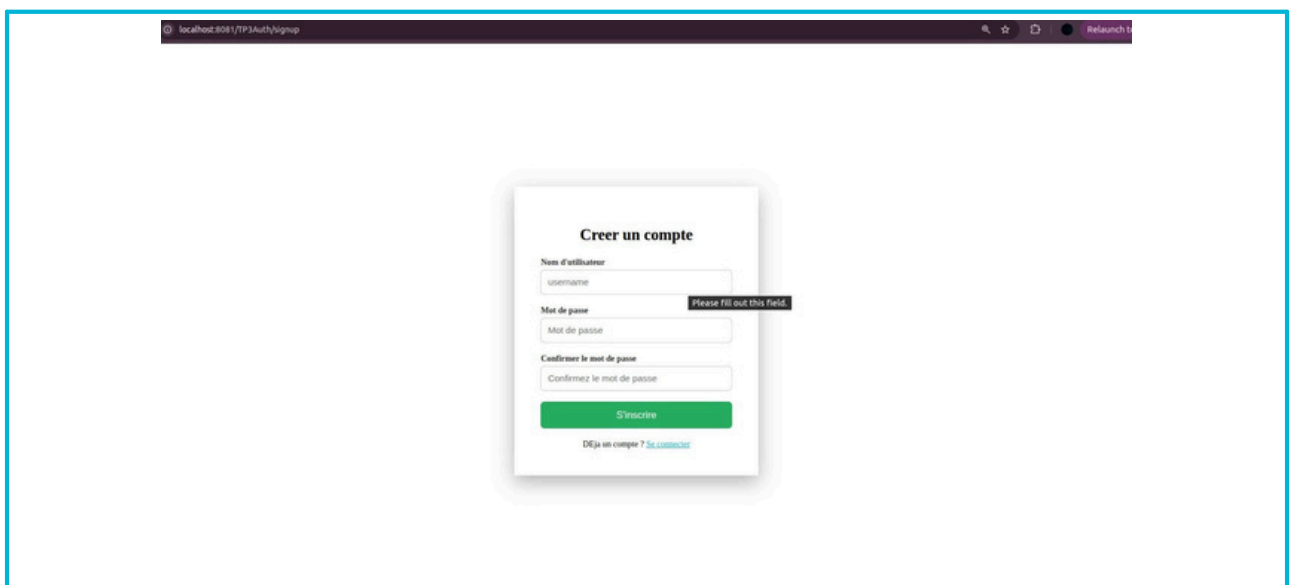
L'application est déployée sur Apache Tomcat 10.1 et accessible à <http://localhost:8081/TP3Auth/>. Les trois pages (Login, Signup, Home) sont pleinement fonctionnelles.

Page de Connexion

The screenshot shows a web browser window with the address bar displaying 'localhost:8081/TP3Auth/login'. The main content area features a centered white card with a light gray border and a subtle shadow. The card is titled 'Connexion' in bold black text, with a subtitle 'Connectez-vous' below it. There are two input fields: 'Nom d'utilisateur' (containing 'username') and 'Mot de passe' (containing 'mot de passe'). Below these fields is a green button labeled 'Se connecter'. At the bottom of the card, there is a link that says 'Pas de compte ? S'inscrire'.

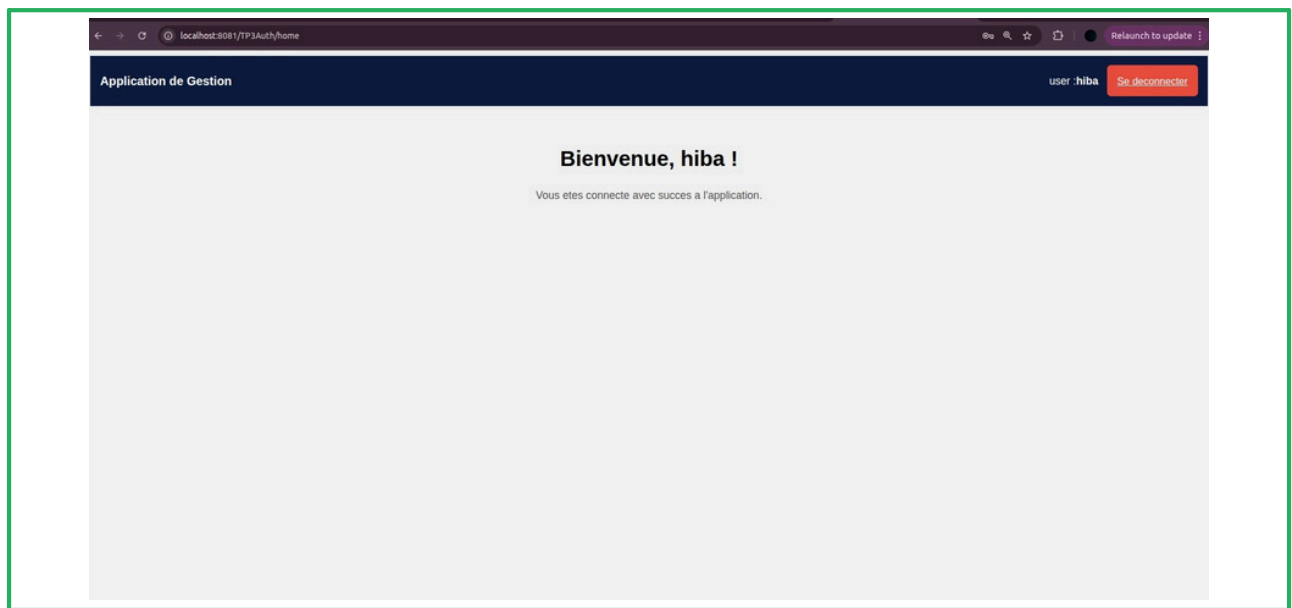
- *formulaire de connexion (login.jsp)*

Page d'Inscription

The screenshot shows a web browser window with the address bar displaying 'localhost:8081/TP3Auth/signup'. The main content area features a centered white card with a light gray border and a subtle shadow. The card is titled 'Créer un compte' in bold black text. There are three input fields: 'Nom d'utilisateur' (containing 'username'), 'Mot de passe' (containing 'Mot de passe' and a red error message 'Please fill out this field.'), and 'Confirmez le mot de passe' (containing 'Confirmez le mot de passe'). Below these fields is a green button labeled 'S'inscrire'. At the bottom of the card, there is a link that says 'Déjà un compte ? Se connecter'.

- *formulaire d'inscription*

Page d'Accueil



- Page accueil après connexion réussie (*home.jsp*) ,session active

La page Home affiche le nom de l'utilisateur connecté (`${username}`) récupéré depuis la session.
La navbar affiche un bouton **Se déconnecter** qui appelle `/logout` pour invalider la session.

Ce TP a permis de maîtriser les mécanismes fondamentaux de la sécurité web en JEE :

gestion des sessions, filtrage des requêtes et stockage en mémoire.

L'application est fonctionnelle avec inscription, connexion sécurisée et déconnexion.

Compétence acquise

- Gestion des sessions HTTP
- Filtre de sécurité
- Base de données en mémoire
- Validation côté serveur
- Navigation sécurisée
- Servlets GET + POST
- RequestDispatcher
- Expression Language JSP

Élément	Détail
Projet	TP3Auth :Dynamic Web Project Eclipse
Serveur	Apache Tomcat 10.1 , Port 8081
Technologies	Jakarta Servlet 6.0 + JSP + Session + Filter
Stockage	ArrayList (en mémoire, sans BDD)
Sécurité	AuthFilter @WebFilter('/home') + session HTTP
Utilisateurs par défaut	admin/1234 - hiba/password
Résultat	Application d'authentification